Compte Rendu

Tp 4

Les principes de la POO :

Héritage, Polymorphisme, Classes abstraites et Interfaces

Année universitaire : 2025-2026

Nom: Hiba Jebabra

Objectif :

L'objectif de ce TP est de comprendre les principes de bases de la programmation orienté objet.

Exercice 1 (*Initialisation et spécialisation d'une entité*):

```java
package ex1;

public class Personne {
    String nom;
    String prenom;
    int age;

    public Personne (String nom,String prenom, int age) {
        this.nom = nom;
        this.prenom = prenom;
        this.age = age;
    }
    public void afficherInformations() {
        System.out.println("nom:"+nom+",prenom:"+prenom+",age="+age
    }
    public boolean estMajeur() {
        return age >= 18;
    }
    public String sePresenter() {
        return "Je m'appelle " + prenom + " " + nom + " et j'ai " +
    }

}
```

Main.java | Etudiant.java | Personne.java

```java
package ex1;

public class Etudiant extends Personne {
    String niveauEtude;
    Double moy;

    public Etudiant(String nom, String prenom, int age, String niveau, d
            super(nom, prenom, age);
            this.niveauEtude = niveau;
            this.moy = moyenne;
    }
    public String calculerMention() {
        if (moy >= 16) return "Très bien";
        else if (moy >= 14) return "Bien";
        else if (moy >= 12) return "Assez bien";
        else return "Ajourné";
        }
    public boolean estAdmis() {
            return moy >= 12;
        }
    public void afficherProfil() {
            afficherInformations();
            System.out.println("Niveau: " + niveauEtude);
            System.out.println("Moyenne: " + moy);
            System.out.println("Mention: " + calculerMention());
            System.out.println("Admis: " + (estAdmis() ? "Oui" : "Non"));
        }

}
```

```java
package ex1;

public class Main {
    public static void main(String[] arg){
        Etudiant E = new Etudiant("hiba","jebabra",20,"premiere annee cyc
        E.afficherProfil();

        System.out.println(E.sePresenter());
    }

}
```

Exécution:

```
<terminated> Main [Java Application] C:\Users\HIBA\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full
nom:hiba,prenom:jebabra,age=20
Niveau: premiere annee cycle
Moyenne: 16.0
Mention: Très bien
Admis: Oui
Je m'appelle jebabra hiba et j'ai 20 ans.
```

Exercice 2 (*Constructeur implicite vs explicite*) :

Voiture.java | Vehicule.java | Main.java

```java
package ex2;

public class Vehicule {
    String marque;

    public Vehicule(String marque ) {
        this.marque = marque;
    }
    public void afficherMarque() {
        System.out.println("marque :"+marque);
    }
    public void demarrer() {
        System.out.println("Le véhicule    "+marque+"   démarre");
    }

    public void arreter() {
        System.out.println("Le véhicule    "+marque+"   s'arrête");
    }
}

```

```java
package ex2;

public class Voiture extends Vehicule {
    int nbPortes ;
    String carburant;
    String color;

    public Voiture(String marque,int nbPortes, String carburant, String colo
        super(marque);
        this.nbPortes = nbPortes;
        this.carburant = carburant ;
        this.color = color;

    }
    public void afficherDetails() {
        System.out.println("la voiture de marque "+marque+"   color:"+color+
    }
    public void Klaxonner() {
        System.out.println("pip pip pipi pipppppp !");
    }
```

```java
package ex2;

public class Main {
    public static void main(String [] args) {
        Voiture v = new Voiture("BMW",4,"Essence","white");

        v.afficherDetails();
        v.Klaxonner();
        v.demarrer();
        v.arreter();
    }

}
```

Exécution:

```
la voiture de marque BMW   color:white   nombres de portes:4     carburant :Essence
pip pip pipi pipppppp !
Le véhicule    BMW   démarre
Le véhicule    BMW   s'arrête
```

Exercice 3 (*héritage – Système bancaire*) :

*Compte.java CompteCourant.java CompteEpargne.java Main.java

```java
package ex3;

public class Compte {
    double solde;
    public Compte(Double solde) {
        this.solde = solde;

    }
    public void deposer(double Montant) {
        solde += Montant;
    }
    public void retirer(double montant) {
        if(montant <= solde) {
            solde -= montant;
        }else {
            System.out.println("solde insuffisant");
        }

    }
    public double consulterSolde() {
            return solde;
        }

    public void afficher() {
            System.out.println("solde :"+solde);
        }

}
```

```java
package ex3;

public class CompteCourant extends Compte {
    double decouvert;

    public CompteCourant(double solde , double decouvert) {
        super(solde);
        this.decouvert = decouvert;
    }

    public boolean autoriserDecouvert(double montant) {
        return (solde-montant)>= - decouvert;
    }
    @Override
    public void retirer(double montant) {
        if(autoriserDecouvert(montant)) {
            solde -= montant;
        }else {
            System.out.println("depassement du decouvert:");
        }
    }

    public double calculerFrais() {
        return solde <0 ? 50:0;
    }

    @Override
    public void afficher() {
        System.out.println("Compte Courant");
        System.out.println("Solde: " + solde);
        System.out.println("Découvert autorisé: " + decouvert);
    }


}
```

```java
package ex3;

public class CompteEpargne extends Compte {
    double taux;

    public CompteEpargne(double solde, double taux) {
        super(solde);
        this.taux = taux;
    }
    public double calculerInterets() {
        return solde*taux;
    }
    public void ajouterInterets() {
        solde += calculerInterets();
    }
    @Override
    public void afficher() {
        System.out.println("Compte Épargne");
        System.out.println("Solde: " + solde);
        System.out.println("Taux: " + taux);
    }
```

```java
package ex3;

public class Main {
    public static void main(String[] args) {
        CompteCourant cc = new CompteCourant(1500,500);
        cc.retirer(2000);
        cc.afficher();
        System.out.println("frais:"+cc.calculerFrais()+"\n");

        CompteEpargne ce = new CompteEpargne(2000, 0.05);
        ce.ajouterInterets();
        ce.afficher();
    }

}
```

Exécution:

```
Compte Courant
Solde: -500.0
Découvert autorisé: 500.0
frais:50.0

Compte Épargne
Solde: 2100.0
Taux: 0.05
```

Exercice 4 (Polymorphisme - annimaux):

Animal.java | chat.java | chien.java | Main.java

```java
package ex4;

public class Animal {

    public void crier() {
        System.out.println("L'animal crie");

    }
    public void manger() {
        System.out.println(" L'animal mange");
    }
    public void dormir() {
        System.out.println("L'animal dort");
    }

}
```

```java
package ex4;

public class chat extends Animal {

    @Override
    public void crier() {
        System.out.println("Le chat miaule ");
    }

    public void ronronner() {
        System.out.println("Le chat ronronne");
    }

    public void grimper() {
        System.out.println("Le chat grimpe");
    }
}
```

```java
package ex4;

public class chien extends Animal {

    @Override
    public void crier() {
        System.out.println("Le chien aboie ");
    }

    public void garder() {
        System.out.println("Le chien garde la maison");
    }

    public void jouer() {
        System.out.println("Le chien joue");
    }
}
```

```java
public class Main {
    public static void main(String[] args) {

        Animal c1 = new chien();
        Animal c2 = new chat();

        c1.crier();
        c2.crier();

        c1.manger();
        c2.manger();

        c1.dormir();
        c2.dormir();

        ((chien)c1).garder();
        ((chien)c1).jouer();
    }
}
```

Exécution :

```
Le chien aboie
Le chat miaule
 L'animal mange
 L'animal mange
L'animal dort
L'animal dort
Le chien garde la maison
Le chien joue
```

Exercice 5 (*Polymorphisme - figures géométriques*):

Figure.java | Triangle.java | Cercle.java | Main.java

```java
package ex5;

public abstract class Figure {
     double x;
     double y;

    public Figure(double x, double y) {
        this.x = x;
        this.y = y;
    }


    public abstract void dessiner();


    public void deplacer(double dx, double dy) {
        x += dx;
        y += dy;
    }


    public abstract void redimensionner(double facteur);
}
```

```java
public class Triangle extends Figure {
    private double a, b, c;

    public Triangle(double x, double y, double a, double b, double c) {
        super(x, y);
        this.a = a;
        this.b = b;
        this.c = c;
    }

    @Override
    public void dessiner() {
        System.out.println("Je dessine un triangle de position (" + x + "
    }

    @Override
    public void redimensionner(double facteur) {
        a *= facteur;
        b *= facteur;
        c *= facteur;
    }

    public double calculerPerimetre() {
        return a + b + c;
    }

    public double calculerSurface() {
        double s = calculerPerimetre() / 2;
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }
}
```

```java
public class Cercle extends Figure {
    private double rayon;

    public Cercle(double x, double y, double rayon) {
        super(x, y);
        this.rayon = rayon;
    }

    @Override
    public void dessiner() {
        System.out.println("Je dessine un cercle de centre (" + x + "," +
    }

    @Override
    public void redimensionner(double facteur) {
        rayon *= facteur;
    }

    public double calculerPerimetre() {
        return 2 * Math.PI * rayon;
    }

    public double calculerSurface() {
        return Math.PI * rayon * rayon;
    }
}
```

```java
ackage ex5;

ublic class Main {
    public static void main(String[] args) {


        Figure[] figures = new Figure[2];

        figures[0] = new Cercle(0, 0, 5);
        figures[1] = new Triangle(1, 1, 3, 4, 5);

        for (int i = 0; i < figures.length; i++) {

            figures[i].dessiner();

            figures[i].deplacer(2, 3);
            figures[i].dessiner();

            figures[i].redimensionner(2);


            if (figures[i] instanceof Cercle) {
                Cercle c = (Cercle) figures[i];
                System.out.println("Périmètre: " + c.calculerPerimetre());
                System.out.println("Surface: " + c.calculerSurface());
            }

            if (figures[i] instanceof Triangle) {
                Triangle t = (Triangle) figures[i];
                System.out.println("Périmètre: " + t.calculerPerimetre());
                System.out.println("Surface: " + t.calculerSurface());
            }


        }
    }
```

Exécution:

```
Je dessine un cercle de centre (0.0,0.0)
Je dessine un cercle de centre (2.0,3.0)
Périmètre: 62.83185307179586
Surface: 314.1592653589793
Je dessine un triangle de position (1.0,1.0)
Je dessine un triangle de position (3.0,4.0)
Périmètre: 24.0
Surface: 24.0
```

Exercice 7 (*Classe abstraite et calculs géométriques*):

Forme.java | Cercle.java | Rectangle.java | Main.java

```java
package ex7;

abstract class Forme {

    public abstract double  calculerSurface ();
    public abstract double calculerPerimetre();
    public abstract boolean validerDimensions();

    void afficherDescription() {
        System.out.println("c est une forme geometrique ");

}
    int comparerSurface(Forme f) {
        return Double.compare(this.calculerSurface(), f.calculerSurface()
    }
}
```

```java
class Cercle  extends Forme{
    double rayon;

    Cercle(double rayon){
        this.rayon = rayon;
    }

    @Override
    public double calculerSurface() {
        return Math.PI*rayon*rayon;
    }

    @Override
    public double calculerPerimetre() {
        return 2 * Math.PI * rayon;
       }

    @Override
    public boolean validerDimensions() {
         return rayon  > 0;
       }


}
```

```java
class Rectangle extends Forme {
    double largeur;
    double hauteur;

    Rectangle(double l,double h){
        this.largeur = l;
        this.hauteur = h;
    }


    @Override
    public double calculerSurface() {
        return largeur * hauteur;
    }

    @Override
    public double calculerPerimetre() {
        return 2 * (largeur + hauteur);
    }

    @Override
    public boolean validerDimensions() {
        return largeur > 0 && hauteur > 0;
    }


}
```

```java
public class Main {
    public static void main(String[] args) {
        Forme r = new Rectangle(5,8);
        Forme c = new Cercle(5);

        r.afficherDescription();
        c.afficherDescription();

        System.out.println("rectangle valide ?" +r.validerDimensions());
        System.out.println("cercle valide ?" +c.validerDimensions());

        System.out.println("surface de rectangle r :" +r.calculerSurface(
        System.out.println("perimetre de rectangle r :" +r.calculerPerime

        System.out.println("Surface Cercle c : " + c.calculerSurface());
        System.out.println("Périmètre Cercle c : " + c.calculerPerimetre(


        int comparaison = r.comparerSurface(c);

        if (comparaison > 0) {
            System.out.println("Le rectangle a une surface plus grande.")
        } else if (comparaison < 0) {
            System.out.println("Le cercle a une surface plus grande.");
        } else {
            System.out.println("Les deux formes ont la même surface.");
        }
    }
```

Exécution:

```
c est une forme geometrique
c est une forme geometrique
rectangle valide ?true
cercle valide ?true
surface de rectangle r :40.0
perimetre de rectangle r :26.0
Surface Cercle c : 78.53981633974483
Périmètre Cercle c : 31.41592653589793
Le cercle a une surface plus grande.
```

Exercice 8 (*Interface*):

```java
package ex8;


public interface Volant {
    void voler();
    void atterrir();
    void changerAltitude(double altitude);
}
```

```java
package ex8;


public class Avion implements Volant {

    private double altitude;
    private int passagers;

    @Override
    public void voler() {
        System.out.println("L'avion décolle et vole");
    }

    @Override
    public void atterrir() {
        System.out.println("L'avion atterrit");
        altitude = 0;
    }

    @Override
    public void changerAltitude(double altitude) {
        this.altitude = altitude;
        System.out.println("Avion à altitude : " + altitude);
    }


    public void embarquerPassagers(int nb) {
        passagers += nb;
        System.out.println(nb + " passagers embarqués");
    }

    public void afficherAltitude() {
        System.out.println("Altitude actuelle : " + altitude);
    }
}
```

```java
public class Oiseau implements Volant {

    private double altitude;

    @Override
    public void voler() {
        System.out.println("L'oiseau vole");
    }

    @Override
    public void atterrir() {
        System.out.println("L'oiseau atterrit");
        altitude = 0;
    }

    @Override
    public void changerAltitude(double altitude) {
        this.altitude = altitude;
        System.out.println("Oiseau à altitude : " + altitude);
    }


    public void migrer() {
        System.out.println("L'oiseau migre");
    }

    public void construireNid() {
        System.out.println("L'oiseau construit un nid");
    }
}
```

```java
package ex8;

public class Main {
    public static void main(String[] args) {

        Oiseau o = new Oiseau();
        Avion a = new Avion();


        o.voler();
        o.changerAltitude(100);
        o.migrer();
        o.construireNid();
        o.atterrir();

        System.out.println("----------------");


        a.voler();
        a.changerAltitude(10000);
        a.embarquerPassagers(150);
        a.afficherAltitude();
        a.atterrir();
    }
}
```

Exécution :

```
L'oiseau vole
Oiseau à altitude : 100.0
L'oiseau migre
L'oiseau construit un nid
L'oiseau atterrit
----------------
L'avion décolle et vole
Avion à altitude : 10000.0
150 passagers embarqués
Altitude actuelle : 10000.0
L'avion atterrit
```

Exercice 9 (*héritage, interface et abstraction*):

Electrique.java | Vehicule.java | VoitureElec... | Main.java

```java
package ex9;

public interface Electrique {
    void charger();
    void verifierBatterie();
}
```

```java
public abstract class Vehicule {
    protected double vitesse;

    public Vehicule() {
        this.vitesse = 0;
    }

    public void accelerer(double valeur) {
        vitesse += valeur;
        System.out.println("la vitesse apres accélération : " + vitesse);
    }

    public void freiner(double valeur) {
        vitesse -= valeur;
        if (vitesse < 0) vitesse = 0;
        System.out.println("  la vitesse apres freinage : " + vitesse);
    }


    public abstract void demarrer();
}
```

```java
public class VoitureElectrique extends Vehicule implements Electrique {

    private int batterie = 100;

    @Override
    public void demarrer() {
        System.out.println("La voiture électrique démarre silencieusement
    }

    @Override
    public void charger() {
        batterie = 100;
        System.out.println("Batterie chargée à 100%");
    }

    @Override
    public void verifierBatterie() {
        System.out.println("Niveau batterie : " + batterie + "%");
    }


    public void afficherAutonomie() {
        System.out.println("Autonomie estimée : " + (batterie * 3) + " km
    }

    public void optimiserConsommation() {
        System.out.println("Mode éco activé");
    }
}
```

```java
public class Moto extends Vehicule {

    @Override
    public void demarrer() {
        System.out.println("La moto démarre avec un bruit fort");
    }


    public void faireRoueArriere() {
        System.out.println("La moto fait une roue arrière !");
    }

    public void afficherTypeMoto() {
        System.out.println("Moto sportive");
    }
```

```java
public class Main {
    public static void main(String[] args) {


        VoitureElectrique v = new VoitureElectrique();
        v.demarrer();
        v.accelerer(50);
        v.verifierBatterie();
        v.afficherAutonomie();
        v.optimiserConsommation();
        v.freiner(20);
        v.charger();




        Moto m = new Moto();
        m.demarrer();
        m.accelerer(80);
        m.faireRoueArriere();
        m.afficherTypeMoto();
        m.freiner(30);
    }
}
```

Exécution :

```
La voiture électrique démarre silencieusement
la vitesse apres accélération : 50.0
Niveau batterie : 100%
Autonomie estimée : 300 km
Mode éco activé
  la vitesse apres freinage : 30.0
Batterie chargée à 100%
La moto démarre avec un bruit fort
la vitesse apres accélération : 80.0
La moto fait une roue arrière !
Moto sportive
  la vitesse apres freinage : 50.0
```

Exercice 10 (*Cas d'étude – Système de gestion d'une bibliothèque numérique*):

```java
public abstract  class Document {
    protected int id;
    protected String titre;
    protected String auteur;
    protected boolean disponible;

    Document (int id ,String titre, String auteur ){
        this.id = id;
        this.titre = titre ;
        this.auteur = auteur;
    }

    void emprunter() {
        if(disponible) {
            disponible = false;
            System.out.println("Document emprunté.");
        }
        else
            System.out.println(" Document non disponible.");

    }
    void retourner() {
        if (!disponible) {
            disponible = true;
            System.out.println("Document retourné.");
        } else {
            System.out.println("Le document est déjà disponible.");
        }
    }

    void afficherInformations() {
        System.out.println("ID: " + id + ", Titre: " + titre + ", Auteur:
    }

    abstract int calculerDureeEmpruntMax();


}
```

Magazine.java | DocumentNume... | Utilisateur... | test.java

```java
package ex10;
public class DocumentNumerique extends Document implements Consultable {
    private String format;
    private double tailleFichier;

    public DocumentNumerique(int id, String titre, String auteur, String
        super(id, titre, auteur);
        this.format = format;
        this.tailleFichier = taille;
    }

    @Override
    public int calculerDureeEmpruntMax() {
        return 0;
    }

    @Override
    public void consulter() {
        System.out.println("Consultation du document numérique.");
    }

    @Override
    public void afficherResume() {
        System.out.println("Résumé du document numérique.");
    }

    public void telecharger() {
        System.out.println("Téléchargement");
    }

    public void afficherTaille() {
        System.out.println("Taille: " + tailleFichier + " MB");
    }

```

```java
package ex10;

import java.time.LocalDate;

class Magazine extends Document implements Empruntable {
    private int numero;
    private LocalDate datePublication;

    Magazine(int id, String titre, String auteur, int numero, LocalDate d
        super(id, titre, auteur);
        this.numero = numero;
        this.datePublication = date;
    }

    @Override
    int calculerDureeEmpruntMax() {
        return 7;
    }

    @Override
    public boolean estDisponible() {
        return disponible;
    }

    @Override
    public void reserver() {
        System.out.println("Magazine réservé.");
    }

    boolean estRecent() {
        return datePublication.isAfter(LocalDate.now().minusMonths(6));
    }

    void afficherEdition() {
        System.out.println("Numéro: " + numero + ", Date: " + datePublica
    }
}
```

```java
class Livre extends Document implements Empruntable {
    private int nombreDePages;
    private String genre;

    Livre(int id, String titre, String auteur, int pages, String genre) {
        super(id, titre, auteur);
        this.nombreDePages = pages;
        this.genre = genre;
    }

    @Override
    int calculerDureeEmpruntMax() {
        return 14;
    }

    @Override
    public boolean estDisponible() {
        return disponible;
    }

    @Override
    public void reserver() {
        System.out.println("Livre réservé.");
    }

    void afficherDetails() {
        afficherInformations();
        System.out.println("Pages: " + nombreDePages + ", Genre: " + genr
    }

    void recommander() {
        System.out.println("Livre recommandé !");
    }
}
```

```java
public interface Consultable {
    void consulter();
    void afficherResume();
}
```

```java
interface Empruntable {
    boolean estDisponible();
    void reserver();
}
```

```java
package ex10;
public class DocumentNumerique extends Document implements Consultable {
    private String format;
    private double tailleFichier;

    public DocumentNumerique(int id, String titre, String auteur, String
        super(id, titre, auteur);
        this.format = format;
        this.tailleFichier = taille;
    }

    @Override
    public int calculerDureeEmpruntMax() {
        return 0;
    }

    @Override
    public void consulter() {
        System.out.println("Consultation du document numérique.");
    }

    @Override
    public void afficherResume() {
        System.out.println("Résumé du document numérique.");
    }

    public void telecharger() {
        System.out.println("Téléchargement");
    }

    public void afficherTaille() {
        System.out.println("Taille: " + tailleFichier + " MB");
    }


}
```

```java
import java.util.ArrayList;

public class Utilisateur {
    private int id;
    private String nom;
    private ArrayList<Document> historique = new ArrayList<>();

    public Utilisateur(int id, String nom) {
        this.id = id;
        this.nom = nom;
    }

    public void emprunterDocument(Document d) {
        d.emprunter();
        historique.add(d);
    }

    public void retournerDocument(Document d) {
        d.retourner();
    }

    public void afficherHistorique() {
        for (Document d : historique) {
            d.afficherInformations();
        }
    }
}
```

Exécution :

```
ID: 1, Titre: Java, Auteur: Auteur A, Disponible: false
Durée max: 14
ID: 2, Titre: Tech, Auteur: Auteur B, Disponible: false
Durée max: 7
ID: 3, Titre: PDF Guide, Auteur: Auteur C, Disponible: false
Durée max: 0
```

```java
package ex10;

import java.util.ArrayList;

public class test {
    public static void main(String[] args) {

        ArrayList<Document> docs = new ArrayList<>();

        docs.add(new Livre(1, "Java", "Ali", 300, "Programmation"));
        docs.add(new Magazine(2, "Tech", "Sara", 10, java.time.LocalDate.now()));
        docs.add(new DocumentNumerique(3, "PDF Guide", "Omar", "PDF", 5.2));


        rechercherParTitre(docs, "Java");


        afficherDisponibles(docs);
    }


    public static void rechercherParTitre(ArrayList<Document> docs, String titre) {
        for (Document d : docs) {
            if (d.titre.equalsIgnoreCase(titre)) {
                d.afficherInformations();
            }
        }
    }


    public static void afficherDisponibles(ArrayList<Document> docs) {
        for (Document d : docs) {
            if (d.disponible) {
                d.afficherInformations();
            }
        }
    }
}
```